Exploring & Pre-processing Immigration Data with Pandas

What I'm Doing

I'm exploring a dataset on **immigration to Canada from 1980 to 2013** (source: UN International Migration Flows — 2015 revision). The goal is to **load**, **clean**, **explore**, and **pre-process** the data using **Pandas** so it's ready for analysis and visualization later.

1. Setting Up

First, I install the Excel reader and import the core libraries.

```
!pip install openpyxl
```

```
import numpy as np
import pandas as pd
```

Then I load the dataset directly from an Excel file hosted online. I skip the first 20 rows (metadata) and the last 2 rows (footnotes).

```
df_can = pd.read_excel(
    'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/'
    'IBMDeveloperSkillsNetwork-DV0101EN-SkillsNetwork/Data%20Files/Canada.xlsx',
    sheet_name='Canada by Citizenship',
    skiprows=range(20),
    skipfooter=2)
df_can.head()
```

	Type	Coverage	OdName	AREA	AreaName	REG	RegName	DEV	DevName	1980
0	Immigrants	Foreigners	Afghanistan	935	Asia	5501	Southern Asia	902	Developing regions	16
1	Immigrants	Foreigners	Albania	908	Europe	925	Southern Europe	901	Developed regions	1
2	Immigrants	Foreigners	Algeria	903	Africa	912	Northern Africa	902	Developing regions	80
3	Immigrants	Foreigners	American Samoa	909	Oceania	957	Polynesia	902	Developing regions	0
4	Immigrants	Foreigners	Andorra	908	Europe	925	Southern Europe	901	Developed regions	0

5 rows × 43 columns

Result: The data is loaded. Let me check what we're working with.

2. Getting to Know the Dataset

I check the structure, columns, index, and dimensions.

```
df_can.info(verbose=False)
```

```
<class 'pandas.DataFrame'>
RangeIndex: 195 entries, 0 to 194
Columns: 43 entries, Type to 2013
dtypes: int64(37), str(6)
memory usage: 65.6 KB
```

Result: Shows column count, data types, and memory usage.

```
df_can.columns
```

```
Index([    'Type', 'Coverage',   'OdName',     'AREA', 'AreaName',      'REG',
        'RegName',      'DEV',  'DevName',       1980,       1981,       1982,
             1983,       1984,       1985,       1986,       1987,       1988,
             1989,       1990,       1991,       1992,       1993,       1994,
             1995,       1996,       1997,       1998,       1999,       2000,
             2001,       2002,       2003,       2004,       2005,       2006,
             2007,       2008,       2009,       2010,       2011,       2012,
             2013],
      dtype='object')
```

```
df_can.index
```

```
RangeIndex(start=0, stop=195, step=1)
```

```
df_can.shape
```

```
(195, 43)
```

Result: `(195, 43)` — 195 countries, 43 columns (type info + years 1980–2013).

Note: The main types stored in *pandas* objects are `float`, `int`, `bool`, `datetime64[ns]`, `datetime64[ns, tz]`, `timedelta[ns]`, `category`, and `object` (string). In addition, these dtypes have item sizes, e.g. `int64` and `int32`.

3. Cleaning the Data

I drop unnecessary columns — `AREA`, `REG`, `DEV`, `Type`, `Coverage` — since they're codes and metadata we don't need.

```
df_can.drop(['AREA', 'REG', 'DEV', 'Type', 'Coverage'], axis=1, inplace=True)
df_can.head(2)
```

	OdName	AreaName	RegName	DevName	1980	1981	1982	1983	1984	1985	...	2004
0	Afghanistan	Asia	Southern Asia	Developing regions	16	39	39	47	71	340	...	2978
1	Albania	Europe	Southern Europe	Developed regions	1	0	0	0	0	0	...	1450

2 rows × 38 columns

Result: The dataframe now has only the useful columns.

Then I rename columns to be more readable.

```
df_can.rename(columns={
    'OdName': 'Country',
    'AreaName': 'Continent',
    'RegName': 'Region'
}, inplace=True)
df_can.columns
```

```
Index([  'Country', 'Continent',    'Region',   'DevName',        1980,
             1981,        1982,        1983,        1984,        1985,
             1986,        1987,        1988,        1989,        1990,
             1991,        1992,        1993,        1994,        1995,
             1996,        1997,        1998,        1999,        2000,
             2001,        2002,        2003,        2004,        2005,
             2006,        2007,        2008,        2009,        2010,
             2011,        2012,        2013],
      dtype='object')
```

I add a `Total` column summing all years for each country.

```
df_can.dtypes # (optional) checking types can avoid non numerical error
df_can['Total'] = df_can.select_dtypes(include='number').sum(axis=1)
df_can['Total'].head()
```

```
0    58639
1    15699
2    69439
3        6
4       15
Name: Total, dtype: int64
```

Result: A new column with total immigrants per country (1980–2013).

I check for missing values across rows and columns.

```
df_can.isnull().sum().sum()
```

```
np.int64(0)
```

Result: Zero null values across all columns — clean dataset.

A quick statistical summary.

```
df_can.describe()
```

	1980	1981	1982	1983	1984	1985	1
count	195.000000	195.000000	195.000000	195.000000	195.000000	195.000000	195.000
mean	508.394872	566.989744	534.723077	387.435897	376.497436	358.861538	441.27
std	1949.588546	2152.643752	1866.997511	1204.333597	1198.246371	1079.309600	1225.576
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000
25%	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.500
50%	13.000000	10.000000	11.000000	12.000000	13.000000	17.000000	18.000
75%	251.500000	295.500000	275.000000	173.000000	181.000000	197.000000	254.000
max	22045.000000	24796.000000	20620.000000	10015.000000	10170.000000	9564.000000	9470.000

8 rows × 35 columns

Result: Shows count, mean, min, max, quartiles for numeric columns. For example, the average annual immigration per country varies across years.

4. Indexing & Selection (Slicing)

Selecting a Column

Method 1 (quick — no spaces in name):

```
df_can.Country    # returns a Series
```

```
0          Afghanistan
1              Albania
2              Algeria
3       American Samoa
4              Andorra
             ...
190          Viet Nam
191     Western Sahara
192              Yemen
193             Zambia
194           Zimbabwe
Name: Country, Length: 195, dtype: str
```

Method 2 (robust — works with multiple columns):

```
df_can[['Country', 1980, 1981, 1982, 1983, 1984, 1985]]   # returns a DataFrame
```

	Country	1980	1981	1982	1983	1984	1985
0	Afghanistan	16	39	39	47	71	340
1	Albania	1	0	0	0	0	0
2	Algeria	80	67	71	69	63	44
3	American Samoa	0	1	0	0	0	0
4	Andorra	0	0	0	0	0	0
...	...	...	...	...	...	...	...
190	Viet Nam	1191	1829	2162	3404	7583	5907
191	Western Sahara	0	0	0	0	0	0
192	Yemen	1	2	1	6	0	18
193	Zambia	11	17	11	7	16	9
194	Zimbabwe	72	114	102	44	32	29

195 rows × 7 columns

Result: A DataFrame showing Country and the first 6 years of data.

The default numeric index (0–194) makes it hard to look up a country by name. I set `Country` as the index.

```
df_can.set_index('Country', inplace=True)
df_can.index.name = None   # optional: remove the label
df_can.head(3)
```

Country	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	1986	...	20(
Afghanistan	Asia	Southern Asia	Developing regions	16	39	39	47	71	340	496	...	34
Albania	Europe	Southern Europe	Developed regions	1	0	0	0	0	0	1	...	12
Algeria	Africa	Northern Africa	Developing regions	80	67	71	69	63	44	69	...	36

3 rows × 38 columns

Result: Now each row is indexed by country name — much easier to query.

Selecting Rows — `loc` vs `iloc`

Let's look at **Japan** in different ways:

Full row:

```
df_can.loc['Japan']
```

```
Continent                 Asia
Region            Eastern Asia
DevName      Developed regions
1980                       701
1981                       756
1982                       598
1983                       309
1984                       246
1985                       198
1986                       248
1987                       422
1988                       324
1989                       494
1990                       379
1991                       506
1992                       605
1993                       907
1994                       956
1995                       826
1996                       994
1997                       924
1998                       897
1999                      1083
2000                      1010
2001                      1092
2002                       806
2003                       817
2004                       973
2005                      1067
2006                      1212
2007                      1250
2008                      1284
2009                      1194
2010                      1168
2011                      1265
2012                      1214
2013                       982
Total                    27707
Name: Japan, dtype: object
```

Result: All columns for Japan — immigrants by year from 1980 to 2013.

```
df_can.iloc[87]    # Japan is at position 87
```

```
Continent                 Asia
Region            Eastern Asia
DevName      Developed regions
1980                       701
1981                       756
Name: Japan, dtype: object
```

Same result by position.

Year 2013 only:

```
df_can.loc['Japan', 2013]
```

```
np.int64(982)
```

Result: The number of immigrants from Japan in 2013.

Years 1980–1985:

```
df_can.loc['Japan', [1980, 1981, 1982, 1983, 1984, 1985]]
```

```
1980    701
1981    756
1982    598
1983    309
1984    246
1984    246
Name: Japan, dtype: int64
```

Result: A Series showing Japan's immigration numbers for those 6 years.

Alternative Method:

```
df_can.iloc[87, [3, 4, 5, 6, 7, 8]]
```

Exercise: Haiti

I apply the same logic to **Haiti**.

```
# 1. Full row
df_can.loc['Haiti']
df_can[df_can.index == 'Haiti'] # alternative
```

```
Continent    Latin America and the Caribbean
Region                               Caribbean
DevName                   Developing regions
1980                                     1666
1981                                     3692
Name: Haiti, dtype: object
```

Result: All immigration data for Haiti (1980–2013).

```
# 2. Year 2000
df_can.loc['Haiti', 2000]
```

```
np.int64(1631)
```

Result: Haiti's immigrant count in the year 2000.

```
# 3. Years 1990-1995
df_can.loc['Haiti', [1990, 1991, 1992, 1993, 1994, 1995]]
```

```
1990    2379
1991    2829
1992    2399
1993    3655
1994    2100
1995    2014
Name: Haiti, dtype: int64
```

Result: Haiti's immigration numbers for each of those 6 years.

5. Converting Column Names to Strings

Having integers as column names can cause confusion (e.g., is `2013` the year or the positional index?). I convert them all to strings.

```
df_can.columns = list(map(str, df_can.columns))
```

Then I create a handy `years` list for easy reference later.

```
years = list(map(str, range(1980, 2014)))
years
```

Result: `['1980', '1981', '1982', ..., '2013']`

Exercise: Create a `year` list for 1990–2013 and extract Haiti's data

```
year = list(map(str, range(1990, 2014))) # 2014 not included
haiti = df_can.loc['Haiti', year]
haiti
```

```
1990    2379
1991    2829
1992    2399
1993    3655
1994    2100
1995    2014
1996    1955
1997    1645
1998    1295
1999    1439
2000    1631
2001    2433
2002    2174
2003    1930
2004    1652
2005    1682
2006    1619
2007    1598
2008    2491
2009    2080
2010    4744
2011    6503
2012    5868
2013    4152
Name: Haiti, dtype: int64
```

Result: A Series with Haiti's immigration numbers for every year from 1990 to 2013.

6. Filtering Based on a Condition

I want data for **Asian countries only**. I create a boolean condition and pass it into the dataframe.

```
condition = df_can['Continent'] == 'Asia' #True/False
df_can[condition].head(5) # limit displaying rows
```

Country	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	1986	...	20(
Afghanistan	Asia	Southern Asia	Developing regions	16	39	39	47	71	340	496	...	34
Armenia	Asia	Western Asia	Developing regions	0	0	0	0	0	0	0	...	2
Azerbaijan	Asia	Western Asia	Developing regions	0	0	0	0	0	0	0	...	3
Bahrain	Asia	Western Asia	Developing regions	0	2	1	1	1	3	0	...	
Bangladesh	Asia	Southern Asia	Developing regions	83	84	86	81	98	92	486	...	41

5 rows × 38 columns

Result: A filtered DataFrame with only countries where Continent is 'Asia'.

I can combine multiple conditions with `&` (and) or `|` (or).

```
df_can[(df_can['Continent'] == 'Asia') & (df_can['Region'] == 'Southern Asia')]
```

Country	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	1986	...	20
Afghanistan	Asia	Southern Asia	Developing regions	16	39	39	47	71	340	496	...	34
Bangladesh	Asia	Southern Asia	Developing regions	83	84	86	81	98	92	486	...	4'
Bhutan	Asia	Southern Asia	Developing regions	0	0	0	0	1	0	0	...	
India	Asia	Southern Asia	Developing regions	8880	8670	8147	7338	5704	4211	7150	...	362
Iran (Islamic Republic of)	Asia	Southern Asia	Developing regions	1172	1429	1822	1592	1977	1648	1794	...	58
Maldives	Asia	Southern Asia	Developing regions	0	0	0	1	0	0	0	...	
Nepal	Asia	Southern Asia	Developing regions	1	1	6	1	2	4	13	...	6
Pakistan	Asia	Southern Asia	Developing regions	978	972	1201	900	668	514	691	...	143
Sri Lanka	Asia	Southern Asia	Developing regions	185	371	290	197	1086	845	1838	...	49

9 rows × 38 columns

Result: Countries in Southern Asia — e.g., India, Pakistan, Bangladesh, etc.

Exercise: Filter for Africa + Southern Africa

```
df_can[(df_can['Continent'] == 'Africa') & (df_can['Region'] == 'Southern Africa')]
```

Country	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	1986	...	2005
Botswana	Africa	Southern Africa	Developing regions	10	1	3	3	7	4	2	...	7
Lesotho	Africa	Southern Africa	Developing regions	1	1	1	2	7	5	3	...	4
Namibia	Africa	Southern Africa	Developing regions	0	5	5	3	2	1	1	...	6
South Africa	Africa	Southern Africa	Developing regions	1026	1118	781	379	271	310	718	...	988
Swaziland	Africa	Southern Africa	Developing regions	4	1	1	0	10	7	1	...	7

5 rows × 38 columns

7. Sorting Values

I sort by the `Total` column in **descending order** to find the top contributors to Canadian immigration.

```
df_can.sort_values(by='Total', ascending=False, axis=0, inplace=True)
top_5 = df_can.head(5)
top_5
```

Country	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	1986	...
India	Asia	Southern Asia	Developing regions	8880	8670	8147	7338	5704	4211	7150	...
China	Asia	Eastern Asia	Developing regions	5123	6682	3308	1863	1527	1816	1960	...
United Kingdom of Great Britain and Northern Ireland	Europe	Northern Europe	Developed regions	22045	24796	20620	10015	10170	9564	9470	...
Philippines	Asia	South-Eastern Asia	Developing regions	6051	5921	5249	4562	3801	3150	4166	...
Pakistan	Asia	Southern Asia	Developing regions	978	972	1201	900	668	514	691	...

5 rows × 38 columns

The top 5 countries that sent the most immigrants to Canada overall (1980–2013).

Exercise: Top 3 countries in 2013

```
df_can.sort_values(by='2013', ascending=False, axis=0, inplace=True)
top3_2013 = df_can['2013'].head(3)
top3_2013
```

```
Country
China          34129
India          33087
Philippines    29544
Name: 2013, dtype: int64
```

That's the full walkthrough. The dataset is now clean, indexed by country, with string column names, a total column, and ready for visualization